
Conception et évaluation d'un système de recommandation de produits pour le e-commerce basé sur les méthodes de similarité et le Deep Metric Learning

*Comparaison entre KNN hybride, Autoencoder et Deep Metric Learning
(Triplet Loss)*

Mariya Derrag

Travail personnel

Année universitaire 2026 – 2027

Résumé

Comment recommander un vêtement à partir d'un autre lorsqu'aucune donnée d'interaction utilisateur-produit n'est disponible ? C'est la question étudiée dans ce travail. Le jeu de données utilisé contient 50 000 produits vestimentaires décrits par 15 variables, mais ne fournit ni historique d'achat, ni clic, ni évaluation individuelle. Le problème est donc abordé comme une tâche de recommandation produit-à-produit fondée sur les caractéristiques des articles.

Trois approches sont étudiées. La première repose sur un modèle KNN hybride combinant une similarité de contenu et une similarité de prix. La deuxième utilise un autoencoder multi-tâches afin d'apprendre une représentation latente des produits à partir d'un objectif de reconstruction. La troisième repose sur le Deep Metric Learning : un réseau neuronal apprend directement un espace d'embedding dans lequel les produits similaires sont rapprochés à l'aide d'une Triplet Loss.

Les modèles sont comparés selon trois critères : la cohérence de catégorie, la cohérence de sous-catégorie et l'écart relatif de prix. Les résultats montrent que le KNN hybride constitue une baseline particulièrement efficace, avec un Category Match@5 de 0,998 et un écart de prix moyen de 0,121. L'autoencoder atteint une forte cohérence de sous-catégorie, mais conserve une moins bonne maîtrise du prix. Le modèle de Deep Metric Learning obtient le meilleur Subcategory Match@5, avec une valeur de 0,996, au prix d'un écart relatif de prix légèrement supérieur à celui du KNN.

Cette étude met en évidence une distinction importante : apprendre à reconstruire correctement les caractéristiques d'un produit ne garantit pas nécessairement l'obtention d'une représentation optimale pour la recommandation. L'objectif d'apprentissage doit être cohérent avec la tâche finale recherchée.

Mots-clés : système de recommandation, recommandation basée sur le contenu, KNN, autoencoder, Deep Metric Learning, Triplet Loss, embeddings de produits.

Table des matières

Résumé	1
Table des matières	2
Introduction générale	4
1 État de l’art et concepts fondamentaux	6
1.1 Systèmes de recommandation	6
1.1.1 Filtrage collaboratif	6
1.1.2 Content-Based Filtering	6
1.1.3 Systèmes hybrides	6
1.2 Mesures de similarité	6
1.2.1 Similarité cosinus	6
1.2.2 Distance euclidienne	7
1.3 K-Nearest Neighbors (KNN)	7
1.4 Autoencoders	7
1.5 Deep Metric Learning	7
1.6 Triplet Loss	8
2 Présentation et analyse du jeu de données	9
2.1 Description générale	9
2.2 Analyse exploratoire	9
2.3 Corrélations	10
2.4 Prétraitement	10
2.4.1 Variables catégorielles	10
2.4.2 Variables numériques	10
3 Méthodologie proposée	11
3.1 Formulation du problème	11
3.2 Approche 1 : KNN hybride	11
3.2.1 Architecture	11
3.2.2 Score final	11
3.3 Approche 2 : Autoencoder	12
3.3.1 Architecture	12
3.3.2 Recherche des voisins	12
3.4 Approche 3 : Neural Embedding avec Triplet Loss	12
3.4.1 Architecture	13
3.4.2 Construction des triplets	13
3.4.3 Fonction de perte	13

4	Protocole expérimental	14
4.1	Configuration	14
4.2	Analyse de l'entraînement	14
4.3	Métriques d'évaluation	15
5	Résultats et discussion	16
5.1	Résultats du KNN hybride	16
5.2	Résultats de l'autoencoder	16
5.2.1	Première version : reconstruction limitée aux variables numériques .	16
5.2.2	Version améliorée : reconstruction multi-tâches	16
5.2.3	Limites observées	16
5.3	Résultats du Neural Embedding (Triplet Loss)	17
5.4	Analyse des embeddings	17
5.5	Comparaison quantitative	18
5.6	Analyse qualitative	18
5.7	Limites expérimentales	18
6	Discussion	20
6.1	KNN hybride	20
6.2	Autoencoder	20
6.3	Neural Embedding (Deep Metric Learning)	20
	Conclusion générale	22
A	Annexes	23
A.1	Extrait du code : construction des triplets à négatifs difficiles	23
A.2	Extrait du code : fonction de perte Triplet Loss	23

Introduction générale

Contexte

Les catalogues en ligne grossissent plus vite que la capacité d'un client à les parcourir. Une boutique qui vendait quelques centaines de références il y a dix ans en propose aujourd'hui des dizaines de milliers. Face à ce volume, chercher devient fastidieux, et trouver relève parfois du hasard. Les systèmes de recommandation existent pour combler cet écart : ils orientent l'utilisateur vers ce qui a des chances de l'intéresser, sans qu'il ait à tout parcourir lui-même.

Problématique

De nombreux systèmes de recommandation exploitent les interactions entre les utilisateurs et les produits, telles que les clics, les achats ou les évaluations. Ces informations permettent notamment de mettre en œuvre des approches de filtrage collaboratif. Toutefois, ces données ne sont pas toujours disponibles. Dans le jeu de données utilisé ici, aucune interaction utilisateur-produit n'est fournie. La question qui structure ce travail en découle directement :

Comment recommander automatiquement des produits similaires à un article donné en utilisant uniquement ses caractéristiques descriptives et commerciales, en l'absence de toute interaction utilisateur-produit ?

Le problème se formule donc comme une tâche de recommandation **produit-à-produit** fondée sur le contenu (*content-based product-to-product recommendation*).

Objectifs

- Explorer et comprendre la structure du jeu de données de 50 000 produits vestimentaires ;
- Construire une baseline de recommandation interprétable (KNN hybride) ;
- Étudier une première approche de Deep Learning fondée sur l'apprentissage non supervisé (autoencoder) ;
- Apprendre des représentations vectorielles de produits directement optimisées pour la similarité (Deep Metric Learning) ;
- Comparer objectivement les différentes méthodes à l'aide de métriques quantitatives reproductibles.

Contributions

Les principales contributions de ce travail sont :

1. La construction d'un système de recommandation hybride combinant caractéristiques catégorielles et numériques (prix) des produits ;
2. L'étude d'un autoencoder multi-tâches pour l'apprentissage d'une représentation latente, et la mise en évidence de ses limites pour la préservation de la similarité sémantique ;
3. La proposition d'un modèle de Deep Metric Learning utilisant des embeddings neuronaux entraînés par Triplet Loss, avec une stratégie de construction de triplets à négatifs difficiles ;
4. Une comparaison expérimentale rigoureuse entre les trois approches, appuyée sur des métriques de cohérence catégorielle et d'écart de prix.

Chapitre 1

État de l’art et concepts fondamentaux

1.1 Systèmes de recommandation

1.1.1 Filtrage collaboratif

Le filtrage collaboratif (*Collaborative Filtering*) part d’une matrice *User-Item* : qui a acheté quoi, qui a cliqué sur quoi. À partir de là, il repère des utilisateurs aux goûts voisins, ou des produits qui reviennent ensemble dans les historiques d’achat. La méthode fonctionne bien, à une condition : disposer d’un volume conséquent d’interactions. Un nouveau produit sans aucun achat, un nouvel utilisateur sans aucun historique, et le système reste aveugle. C’est le problème classique du *cold start*.

1.1.2 Content-Based Filtering

Le filtrage basé sur le contenu prend le problème autrement. Il recommande un produit à partir de ce qu’il est, catégorie, matière, couleur, prix, et non de qui l’a acheté. Aucun historique utilisateur requis. C’est précisément le cadre de ce travail : le jeu de données ne contient aucune interaction, donc aucun autre choix n’était vraiment envisageable.

1.1.3 Systèmes hybrides

Entre les deux, les systèmes hybrides combinent plusieurs sources, contenu, comportement, contexte, pour compenser ce que chaque approche rate seule.

1.2 Mesures de similarité

1.2.1 Similarité cosinus

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} \quad (1.1)$$

1.2.2 Distance euclidienne

$$d(A, B) = \sqrt{\sum_i (A_i - B_i)^2} \quad (1.2)$$

Ce travail retient la similarité cosinus. Elle ignore l'échelle des vecteurs et ne regarde que leur orientation relative, ce qui tombe bien : dans un espace d'embeddings de grande dimension, c'est précisément cette orientation qui porte le sens.

1.3 K-Nearest Neighbors (KNN)

Le principe du KNN consiste à calculer, pour un produit cible, sa similarité avec l'ensemble des autres produits du catalogue, puis à retenir les K plus proches voisins :

$$\text{Produit cible} \rightarrow \text{Calcul de similarité} \rightarrow \text{Top-}K \text{ voisins} \quad (1.3)$$

Avantages : simplicité, interprétabilité, absence de phase d'entraînement complexe.

Limites : dépendance forte au choix et à la pondération manuelle des caractéristiques ; difficulté à capturer des relations non linéaires entre variables.

1.4 Autoencoders

Un autoencoder est un réseau de neurones entraîné à reconstruire son entrée à travers un goulot d'étranglement (*bottleneck*) de dimension réduite :

$$X \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{X} \quad (1.4)$$

avec pour objectif la minimisation de l'erreur de reconstruction :

$$\min \|X - \hat{X}\|^2 \quad (1.5)$$

Un point mérite d'être posé dès maintenant, il reviendra au chapitre 5 : **bien reconstruire ne veut pas dire bien représenter**. Rien, dans l'objectif de reconstruction, n'oblige deux produits proches en réalité à finir proches dans l'espace latent. Le lien peut exister. Rien ne le garantit.

1.5 Deep Metric Learning

Le Deep Metric Learning vise à apprendre directement une fonction d'embedding telle que :

$$\text{produits similaires} \Rightarrow \text{embeddings proches} \quad (1.6)$$

$$\text{produits différents} \Rightarrow \text{embeddings éloignés} \quad (1.7)$$

Contrairement à l'autoencoder, l'objectif d'entraînement porte directement sur la structure de similarité recherchée, et non sur une tâche proxy (la reconstruction).

1.6 Triplet Loss

La Triplet Loss repose sur trois éléments par exemple d'entraînement :

- un **Anchor** (A) : le produit de référence ;
- un **Positive** (P) : un produit similaire à l'anchor ;
- un **Negative** (N) : un produit dissimilaire à l'anchor.

La fonction de perte est définie par :

$$\mathcal{L} = \max(d(A, P) - d(A, N) + \alpha, 0) \quad (1.8)$$

où α est la marge (*margin*), un hyperparamètre contrôlant l'écart minimal exigé entre la distance positive et la distance négative.

Chapitre 2

Présentation et analyse du jeu de données

2.1 Description générale

Le jeu de données utilisé contient 50 000 produits vestimentaires féminins décrits par 15 variables.

TABLE 2.1 – Variables du jeu de données

Variable	Type	Description
product_id	Catégorielle	Identifiant unique du produit
category	Catégorielle	Catégorie principale (ex. Jeans, Dresses)
subcategory	Catégorielle	Sous-catégorie (ex. Skinny Jeans)
material	Catégorielle	Matière (ex. Denim, Coton)
color	Catégorielle	Couleur
size	Catégorielle	Taille
season	Catégorielle	Saison
style	Catégorielle	Style vestimentaire
price_usd	Numérique	Prix en dollars
discount_percent	Numérique	Pourcentage de réduction
rating	Numérique	Note moyenne
review_count	Numérique	Nombre d'avis
stock_quantity	Numérique	Quantité en stock
units_sold	Numérique	Nombre d'unités vendues
return_rate	Numérique	Taux de retour

2.2 Analyse exploratoire

L'analyse exploratoire couvre six axes :

- la distribution du nombre de produits par catégorie et sous-catégorie ;
- la distribution du prix (`price_usd`) ;
- la distribution des notes (`rating`) ;
- la relation entre `units_sold` et `review_count` ;

- le taux de retour moyen selon la catégorie ;
- le prix moyen par catégorie.

2.3 Corrélations

Une matrice de corrélation entre variables numériques permet d'identifier les relations linéaires, en particulier entre `units_sold` et `review_count`.

2.4 Prétraitement

2.4.1 Variables catégorielles

Deux encodages distincts sont utilisés selon le modèle :

- `TfidfVectorizer` / `OneHotEncoder` pour le modèle KNN classique, appliqué sur une chaîne de caractéristiques combinées ;
- `LabelEncoder` pour les modèles neuronaux, en entrée de couches `Embedding` dédiées à chaque variable catégorielle.

2.4.2 Variables numériques

Les variables numériques sont standardisées :

$$x' = \frac{x - \mu}{\sigma} \quad (2.1)$$

via `StandardScaler`, afin d'homogénéiser leur échelle avant leur intégration dans les réseaux de neurones.

Chapitre 3

Méthodologie proposée

3.1 Formulation du problème

Chaque produit i est représenté par un vecteur de caractéristiques :

$$x_i = [\text{category}, \text{subcategory}, \text{material}, \text{color}, \text{season}, \text{style}, \text{size}, \text{price}, \text{rating}, \dots] \quad (3.1)$$

L'objectif est de déterminer, pour un produit i , l'ensemble des K produits les plus similaires :

$$\text{Top-}K(i) = \arg \max_{j \neq i} \text{Similarity}(i, j) \quad (3.2)$$

3.2 Approche 1 : KNN hybride

Dans ce travail, le terme KNN hybride désigne une méthode de recherche des K produits présentant les scores de similarité les plus élevés. La méthode ne repose pas sur une distance unique appliquée directement aux observations : elle combine une mesure de similarité de contenu et une mesure de proximité des prix.

3.2.1 Architecture

Le modèle combine deux sources de similarité :

- une **similarité de contenu** calculée par distance cosinus sur les caractéristiques catégorielles encodées (One-Hot) via un modèle `NearestNeighbors` ;
- une **similarité de prix**, calculée par rapport à l'écart relatif au prix du produit cible.

3.2.2 Score final

$$\text{FinalScore} = w_1 \cdot \text{ContentSimilarity} + w_2 \cdot \text{PriceSimilarity} \quad (3.3)$$

avec $w_1 = 0,75$, $w_2 = 0,25$, et $w_1 + w_2 = 1$.

La similarité de prix est calculée relativement au prix du produit cible (et non à l'étendue globale du catalogue), afin qu'un même écart absolu soit interprété différemment

selon le segment de prix considéré :

$$\text{PriceSimilarity} = \max\left(0, 1 - \frac{|price_j - price_i|}{\max(price_i, 1)}\right) \quad (3.4)$$

Listing 3.1 – Extrait du modèle KNN hybride

```

1 recommendations["price_similarity"] = 1 - (
2     abs(recommendations["price_usd"] - target_price)
3     / max(target_price, 1)
4 )
5 recommendations["price_similarity"] = recommendations[
6     "price_similarity"
7 ].clip(lower=0)
8
9 recommendations["final_score"] = (
10     0.75 * recommendations["content_similarity"]
11     + 0.25 * recommendations["price_similarity"]
12 )

```

3.3 Approche 2 : Autoencoder

3.3.1 Architecture

$$X \rightarrow \text{Encoder} \rightarrow \text{Embedding } 32D \rightarrow \text{Decoder} \rightarrow \hat{X} \quad (3.5)$$

L'encodeur combine des couches **Embedding** par variable catégorielle et une entrée numérique standardisée, concaténées puis réduites via des couches denses régularisées (L2, Dropout, BatchNormalization) jusqu'au *bottleneck*.

Le décodeur reconstruit séparément chaque variable catégorielle (têtes **softmax**, une par variable) et les variables numériques (tête linéaire, perte MSE) :

$$\mathcal{L}_{\text{total}} = \sum_{c \in \text{cat}} \lambda_c \cdot \text{CE}(x_c, \hat{x}_c) + \lambda_{\text{num}} \cdot \text{MSE}(x_{\text{num}}, \hat{x}_{\text{num}}) \quad (3.6)$$

3.3.2 Recherche des voisins

Une fois entraîné, l'embedding produit (z_i , 32 dimensions) est extrait pour chaque produit, puis la recommandation s'effectue par similarité cosinus directe entre embeddings.

3.4 Approche 3 : Neural Embedding avec Triplet Loss

C'est le modèle central de ce travail, relevant du *Deep Metric Learning*.

3.4.1 Architecture

Category -> Embedding
 Subcategory -> Embedding
 Material -> Embedding
 Color -> Embedding
 Season -> Embedding
 Style -> Embedding
 Size -> Embedding

Variables numeriques
 |
 Concatenation
 |
 Dense 128
 |
 Batch Normalization
 |
 Dropout
 |
 Dense 64
 |
 Product Embedding (32D)
 |
 L2 Normalization

FIGURE 3.1 – Architecture de la tour d’embedding produit

Les trois branches (anchor, positive, negative) partagent les mêmes poids (*shared tower*), garantissant que la même fonction d’embedding est apprise pour tous les produits.

3.4.2 Construction des triplets

- **Anchor** (A) : produit tiré aléatoirement.
- **Positive** (P) : produit de même **category** et même **subcategory** que l’anchor.
- **Negative** (N) : deux stratégies ont été comparées :
 - *Easy Negative* : produit de **category** différente ;
 - *Hard Negative* : produit de même **category** mais de **subcategory** différente.

La version finale retenue utilise un ratio de **100% de négatifs difficiles**, après qu’une première configuration à 70% de négatifs difficiles s’est révélée insuffisante pour discriminer finement les sous-catégories (voir Chapitre 5).

3.4.3 Fonction de perte

$$\mathcal{L} = \max(\|A - P\|^2 - \|A - N\|^2 + \alpha, 0) \quad (3.7)$$

Chapitre 4

Protocole expérimental

4.1 Configuration

TABLE 4.1 – Hyperparamètres du modèle Neural Embedding (Triplet Loss) – version finale

Paramètre	Valeur
Nombre de produits	50 000
Nombre de triplets d'entraînement	200 000
Dimension embedding catégoriel	8
Dimension embedding produit final	32
Taille de batch	512
Optimiseur	Adam
Taux d'apprentissage initial	10^{-3}
Marge (α)	0,1
Ratio de négatifs difficiles	100%
Nombre maximal d'époques	40
Régularisation	EarlyStopping, ReduceLROnPlateau

4.2 Analyse de l'entraînement

La courbe de la Triplet Loss montre une décroissance nette et continue au fil des époques, passant de 0,0525 à la première époque à $5,97 \times 10^{-5}$ en fin d'entraînement.

Une faible valeur de la Triplet Loss indique que le modèle satisfait majoritairement les contraintes définies lors de la construction des triplets. Elle ne constitue cependant pas, à elle seule, une mesure directe de la qualité des recommandations. C'est pourquoi l'évaluation finale repose sur des métriques indépendantes, calculées à partir des voisins effectivement recommandés.

4.3 Métriques d'évaluation

Category Match@K

$$\text{CategoryMatch@K} = \frac{1}{K} \sum_{j=1}^K \mathbb{1}(category_j = category_{\text{target}}) \quad (4.1)$$

Subcategory Match@K

$$\text{SubcategoryMatch@K} = \frac{1}{K} \sum_{j=1}^K \mathbb{1}(subcategory_j = subcategory_{\text{target}}) \quad (4.2)$$

Écart relatif de prix

$$\text{PriceGap} = \frac{|price_{\text{target}} - price_{\text{recommended}}|}{price_{\text{target}}} \quad (4.3)$$

Ces métriques sont calculées sur un échantillon de 200 produits tirés aléatoirement du catalogue, avec $K = 5$ recommandations par produit.

Chapitre 5

Résultats et discussion

5.1 Résultats du KNN hybride

Le KNN hybride sert de référence pour tout le reste. Rien de spectaculaire dans sa conception, mais les résultats parlent : cohérence catégorielle quasi parfaite, écart de prix bien tenu, ce dernier point n'étonnant guère puisqu'il est directement intégré dans le score. Une baseline qui fait le travail qu'on attend d'elle.

5.2 Résultats de l'autoencoder

5.2.1 Première version : reconstruction limitée aux variables numériques

La première version n'a été entraînée qu'à reconstruire les variables numériques. Résultat : un embedding qui ne discrimine rien. Les similarités cosinus s'entassent près de 1,0, peu importe de quels produits il s'agit, et les recommandations en découlent, incohérentes, une robe conseillée pour un jean, par exemple. L'espace latent s'est effondré sur lui-même, sourd à tout ce qui touche à la catégorie du produit.

5.2.2 Version améliorée : reconstruction multi-tâches

Une seconde version corrige le tir : chaque variable catégorielle reçoit sa propre tête `softmax`, avec des poids de perte proportionnels au nombre de classes ($\lambda_c = \log(1 + n_c)$). L'effondrement disparaît, la similarité cosinus moyenne retombe autour de 0,008, et l'exactitude de reconstruction catégorielle atteint 1,000 sur les sept variables. Un vrai progrès.

5.2.3 Limites observées

Mais il a un coût. La tête numérique, écrasée dans le gradient total par les sept têtes catégorielles, n'apprend presque rien du prix. Multiplier son poids par dix ne change presque rien à l'écart de prix dans les recommandations, alors même que la perte de reconstruction numérique, elle, baisse nettement. Autrement dit : le décodeur devient meilleur pour reconstruire le prix sur le papier, sans que le *bottleneck* n'organise réellement son espace autour de cette variable. Le prix reste un signal marginal dans la géométrie de

l'embedding, quoi qu'on fasse des poids. Deux choses différentes, donc : reconstruire, et représenter.

$$\boxed{\text{Qualité de reconstruction} \neq \text{Qualité de recommandation}} \quad (5.1)$$

Les résultats de reconstruction et les résultats de recommandation doivent être interprétés séparément. Une amélioration de la perte de reconstruction ne se traduit pas nécessairement par une amélioration de la structure de voisinage dans l'espace latent.

5.3 Résultats du Neural Embedding (Triplet Loss)

TABLE 5.1 – Évolution des stratégies d'entraînement du modèle Neural Embedding

Configuration	Marge α	Négatifs difficiles
Version initiale	0,3	70%
Version finale	0,1	100%

Une première configuration ($\alpha = 0,3$, 70% de négatifs difficiles) sépare parfaitement les catégories, mais échoue à l'intérieur de chacune : 0,542 seulement sur *Subcategory Match@5*. La raison tient en une ligne : la marge est satisfaite dès que la séparation inter-catégorie est acquise, et le réseau n'a alors plus aucune raison de creuser plus finement.

La première configuration permettait une bonne séparation entre les grandes catégories, mais restait insuffisante pour discriminer finement les sous-catégories. Le passage à des négatifs systématiquement difficiles, c'est-à-dire appartenant à la même catégorie mais à une sous-catégorie différente, a renforcé cette discrimination fine.

La version finale change deux paramètres, la marge, resserrée à $\alpha = 0,1$, et le ratio de négatifs, poussé à 100% de négatifs difficiles (même catégorie, sous-catégorie différente). Cette fois le comportement se corrige : le modèle obtient la meilleure cohérence de sous-catégorie des trois approches, tandis que le KNN hybride conserve un léger avantage pour la cohérence de catégorie et la proximité des prix.

5.4 Analyse des embeddings

Deux chiffres résument l'état de l'embedding final, en dimension 32 : une similarité cosinus moyenne de 0,7642 entre paires aléatoires, et un écart-type de 0,1779.

Comparé à la première version de l'autoencoder, où les similarités s'agglutinaient près de 1,0 avec un écart-type proche de zéro, signe classique d'effondrement, cet écart-type substantiel indique que le modèle discrimine réellement les produits entre eux. La similarité de fond plus élevée qu'en début de réglage n'a rien d'inquiétant : avec 100% de négatifs difficiles, la majorité des comparaisons se font à l'intérieur d'une même catégorie, et l'espace se resserre logiquement autour de familles de produits proches, tous les jeans avec tous les jeans, sans pour autant s'effondrer globalement.

5.5 Comparaison quantitative

TABLE 5.2 – Comparaison des trois approches (moyenne sur 200 produits, $K = 5$)

Modèle	Category Match@5	Subcategory Match@5	Price Gap ↓
KNN hybride	0,998	0,928	0,121
Autoencoder (v3)	0,997	0,993	0,396
Neural Embedding (Triplet Loss)	0,993	0,996	0,155

Le tableau 5.2 présente les résultats moyens obtenus sur 200 produits sélectionnés aléatoirement, avec $K = 5$ recommandations.

Le KNN hybride obtient la meilleure cohérence au niveau de la catégorie (0,998) ainsi que le plus faible écart relatif de prix (0,121). Ce résultat est cohérent avec sa fonction de score, dans laquelle la proximité de prix est explicitement intégrée.

L’autoencoder corrigé atteint une forte cohérence de sous-catégorie (0,993), mais son écart de prix reste nettement plus élevé (0,396). Cela suggère que la représentation latente obtenue préserve efficacement certaines relations catégorielles sans organiser l’espace latent autour de la proximité tarifaire.

Le Neural Embedding obtient le meilleur résultat pour la cohérence de sous-catégorie, avec un Subcategory Match@5 de 0,996. En revanche, son écart relatif de prix (0,155) reste supérieur à celui du KNN. Ce compromis est cohérent avec la fonction d’entraînement utilisée : la Triplet Loss optimise la proximité entre produits définis comme similaires, sans intégrer explicitement le prix dans son objectif.

Note méthodologique : par rigueur, une évaluation sur un ensemble de produits totalement disjoint de ceux utilisés dans la génération des triplets d’entraînement serait à privilégier dans une itération future, afin d’exclure tout effet de mémorisation.

5.6 Analyse qualitative

Pour le produit WC000001 (Jean skinny, denim, rouge, style *Business Casual*), les trois modèles produisent des profils de recommandation distincts :

- le **KNN hybride** recommande exclusivement des *Skinny Jeans* en denim, à des prix très proches de l’article cible ;
- l’**autoencoder** (version corrigée) recommande des produits de sous-catégories cohérentes mais à des prix parfois très éloignés ;
- le **Neural Embedding** recommande majoritairement des jeans de sous-catégories très proches (*Skinny*, *Straight*, *Wide-Leg*), avec un écart de prix modéré.

5.7 Limites expérimentales

La principale limite de cette étude concerne la séparation entre les données utilisées pour construire les triplets et celles utilisées pour l’évaluation. Les produits évalués appartiennent au même catalogue que celui utilisé lors de l’entraînement. Même si le modèle n’est pas entraîné à mémoriser explicitement les recommandations finales, cette configuration ne permet pas d’exclure complètement un effet de mémorisation.

Une amélioration naturelle consisterait à diviser le catalogue en ensembles d'entraînement, de validation et de test disjoints avant la génération des triplets. Les performances finales seraient alors mesurées uniquement sur des produits de test non utilisés dans l'entraînement.

Une seconde limite concerne la nature du dataset. Les données ne contiennent aucune interaction réelle entre utilisateurs et produits. Les métriques utilisées évaluent donc la cohérence structurelle des recommandations, et non leur capacité à prédire les préférences réelles d'utilisateurs.

Chapitre 6

Discussion

6.1 KNN hybride

Le KNN hybride constitue une baseline performante et facilement interprétable. Son principal avantage réside dans la transparence de son fonctionnement : la contribution de la similarité de contenu et de la proximité de prix est explicitement contrôlée par les poids w_1 et w_2 .

Sa principale limite est que ces poids sont définis manuellement et ne sont pas appris à partir des données. La représentation utilisée ne modélise pas non plus directement des relations complexes ou non linéaires entre les caractéristiques.

Fixé à la main, c'est justement sa limite. Les poids w_1 et w_2 ne sont pas appris, ils sont choisis. Et la similarité de contenu repose sur une représentation figée, one-hot ou TF-IDF, incapable de saisir la moindre relation non linéaire entre caractéristiques.

6.2 Autoencoder

Il apprend seul sa représentation, réduit la dimension du problème, et n'a besoin d'aucune étiquette de similarité pour s'entraîner. L'autoencoder permet d'apprendre automatiquement une représentation compacte des produits. Cependant, son objectif principal reste la reconstruction des variables d'entrée. Les expériences réalisées montrent qu'une amélioration de la qualité de reconstruction ne garantit pas une amélioration équivalente de la qualité des recommandations.

La difficulté principale observée concerne l'équilibre entre la reconstruction des variables catégorielles et celle des variables numériques. Les différentes têtes de sortie n'ont pas la même nature ni la même échelle de perte, ce qui rend leur optimisation conjointe délicate.

6.3 Neural Embedding (Deep Metric Learning)

Le modèle de Deep Metric Learning est celui qui optimise le plus directement la notion de proximité entre les produits. Contrairement à l'autoencoder, il ne cherche pas à reconstruire les variables d'entrée, mais à organiser les produits dans un espace vectoriel en fonction des relations définies par les triplets.

Les résultats montrent une excellente cohérence au niveau de la sous-catégorie. Cette performance doit toutefois être interprétée en tenant compte de la stratégie de génération

des triplets. Les produits positifs et négatifs sont définis à partir des informations de catégorie et de sous-catégorie. Une partie de la structure évaluée est donc directement liée au signal utilisé pendant l'entraînement.

Reste enfin une limite qui dépasse ce seul modèle : le prix est absent de la fonction de perte, et donc plus mal maîtrisé que par le KNN. Les métriques de pertinence, quant à elles, sont construites à partir des caractéristiques du dataset lui-même, faute d'interactions utilisateur réelles pour les valider de l'extérieur.

Conclusion générale

Ce travail avait pour objectif de concevoir un système de recommandation produit-à-produit pour un catalogue de vêtements contenant 50 000 articles, en l'absence de toute donnée d'interaction entre utilisateurs et produits.

Trois approches ont été étudiées. Le KNN hybride a constitué une première base-line fondée sur la combinaison d'une similarité de contenu et d'une similarité de prix. Cette méthode a obtenu d'excellents résultats en matière de cohérence de catégorie et de proximité tarifaire, tout en conservant une forte interprétabilité.

L'autoencoder a ensuite été utilisé pour apprendre automatiquement une représentation latente des produits. Les différentes expériences ont montré que la qualité de reconstruction ne constitue pas une garantie directe de la qualité des recommandations. La version multi-tâches améliore la cohérence des voisins obtenus, mais la proximité des prix reste insuffisamment prise en compte dans la structure de l'espace latent.

Enfin, une approche de Deep Metric Learning basée sur la Triplet Loss a été développée. L'utilisation de négatifs difficiles a permis d'améliorer la discrimination entre les sous-catégories. Le modèle final obtient le meilleur résultat pour le Subcategory Match@5, avec une valeur de 0,996. Le KNN hybride reste néanmoins légèrement supérieur pour la cohérence de catégorie et pour la proximité des prix.

Les résultats montrent qu'il n'existe pas de modèle universellement meilleur selon tous les critères. Le choix dépend de la notion de similarité que l'on souhaite privilégier. Une méthode basée sur des règles explicites peut rester très compétitive lorsqu'elle correspond directement aux critères d'évaluation, tandis qu'un modèle neuronal permet d'apprendre des relations plus complexes lorsque la fonction d'entraînement est correctement alignée avec la tâche finale.

Perspectives

- Évaluation sur un ensemble de test totalement séparé des données utilisées pour l'entraînement, afin de renforcer la validité expérimentale des résultats ;
- Intégration de véritables interactions utilisateur-produit et mise en œuvre d'un filtrage collaboratif ou d'un *Neural Collaborative Filtering* ;
- Système hybride combinant filtrage basé sur le contenu et filtrage collaboratif ;
- Intégration d'un objectif de prix directement dans la fonction de perte du Triplet Loss (perte combinée) ;
- Exploitation d'images produits via des architectures CNN ou des modèles multimodaux de type CLIP ;
- Prise en compte du comportement utilisateur pour une recommandation réellement personnalisée ;
- Développement d'une interface web de démonstration du système.

Annexe A

Annexes

A.1 Extrait du code : construction des triplets à négatifs difficiles

```
1 def build_triplets_hard_v2(n_triplets):
2     anchors = rng.integers(0, n_samples, size=n_triplets)
3     positives = np.empty(n_triplets, dtype=int)
4     negatives = np.empty(n_triplets, dtype=int)
5
6     for i, a in enumerate(anchors):
7         key = (category_arr[a], subcategory_arr[a])
8         same_group = group_to_indices[key]
9         p = rng.choice(same_group) if len(same_group) > 1 else a
10        positives[i] = p
11
12        same_cat = category_to_indices[category_arr[a]]
13        n = rng.choice(same_cat)
14        while subcategory_arr[n] == subcategory_arr[a]:
15            n = rng.choice(same_cat)
16        negatives[i] = n
17
18    return anchors, positives, negatives
```

A.2 Extrait du code : fonction de perte Triplet Loss

```
1 def triplet_loss(y_true, y_pred, embed_dim=32, margin=0.1):
2     a = y_pred[:, :embed_dim]
3     p = y_pred[:, embed_dim:2*embed_dim]
4     n = y_pred[:, 2*embed_dim:]
5     pos_dist = tf.reduce_sum(tf.square(a - p), axis=1)
6     neg_dist = tf.reduce_sum(tf.square(a - n), axis=1)
7     return tf.reduce_mean(tf.maximum(pos_dist - neg_dist + margin,
8                                     0.0))
```